Persistence in iOS

Overview

There are **three** common methods to persisting data to device in iOS:

- **UserDefaults** (`NSUserDefaults`)
- **Writing/Reading Files** (`FileManager`)
- **CoreData** (xcdatamodel)

UserDefaults

- Simple key-value data structure provided by **Foundation**
- Used for storing a user's preferences
- Should **not** be used to store
  - Sensitive data
  - Large amounts of data
- **Cached in-memory** for fast access
- We will use the global singleton instance `UserDefaults.standard`

UserDefaults

- Supports the basic Property List data types (and their **Swift** counterparts):
  - **NSString**
  - **NSNumber** (for boolean, integer, double)
  - **NSDate**
  - **NSArray**
  - **NSDictionary**
  - **NSData**

Reading and Writing Files

- **NSData**, **NSArray**, **NSDictionary**, and **NSString** all know how to **write/archive** themselves to disk:

```
func write(to url: URL, atomically: Bool) -> Bool
```

- They also know how to **read/unarchive** themselves from disk with a specific init:

```
init?(contentsOf url: URL)
```

- The above function calls are commonly wrapped in `FileManager` extensions to assist with file location management

FileManager

- A convenient interface to the contents of the file system
- Typically the primary means of interacting with that file system
- Can be used to perform the following actions on files and directories:
  - **Locate**
  - **Create**
  - **Copy**
  - **Move**
  - **Delete**
- May also be used to get information about a file or directory or change some of its attributes
- We will use the default singleton instance `FileManager.default`

URL

- Identifies the **location** of a resource, for example:
    - The path to a **local file**

```
/**
`func url(for directory:, in domain: , appropriateFor url:, create shouldCreate:) throws -> URL`

Locates and optionally creates the specified common directory in a domain

- Parameters:
   - directory: The search path directory.
      The supported values are described in `enum FileManager.SearchPathDirectory`.
   - domain: The file system domain to search.
      The supported values are described in `enum FileManager.SearchPathDomainMask`.
   - url: The file `URL` used to determine the location of the returned `URL`.
      This parameter is ignored unless the directory parameter contains the value
      `.itemReplacementDirectory` and the domain parameter contains the value `.userDomainMask`.
   - shouldCreate: `Bool` value specifying whether or not to create the directory if it does not already exist.
*/

/// Various user-visible documentation, support, and configuration files (/Library).
let libraryDirectory: FileManager.SearchPathDirectory = .libraryDirectory

/// The user's home directory - the place to install user's personal items (~).
let userDomain: FileManager.SearchPathDomainMask = .userDomainMask

do {
    let libraryDirectoryURL: URL = try FileManager.default.url(
        for: libraryDirectory,
        in: userDomain,
        appropriateFor: nil,
        create: true
    )
} catch {
    print(error)
}
```

CoreData

- Wrapper around a relational database
    - Almost always SQLite
- **Managed Object Graph**
    - Work with a graph/tree of objects
    - No need to make SQL queries directly
- Performs many optimizations, including:
    - Maintaining an **in-memory cache**
    - **Minimize** the amount of **disk reads/writes**
    - etc…

Managed Object Model

- Much of **CoreData's** functionality depends on the schema supplied to describe:
    - An application's entities
    - Their properties
    - The relationships between them
- A managed object model allows **CoreData** to map from records in a persistent store to managed objects used in an application
- The source file of this model will have the extension `.xcdatamodeld`

NSPersistentContainer

- A container that encapsulates the CoreData stack in an application
- Provides access to:
    - The **managed object model**
        - The source file containing the extension `.xcdatamodeld`
    - The **managed object context**
        - Via `viewContext:` an `NSManagedObjectContext` associated with the **main queue** of an application
        - Methods for creating/ accessing background contexts to perform asynchronous tasks:

```
func newBackgroundContext() -> NSManagedObjectContext
func performBackgroundTask(_ block: @escaping (NSManagedObjectContext) -> Void)
```

    - The **persistent store coordinator**
        - An object that uses the model to help contexts and persistent stores, i.e. the **SQLite** file, communicate

NSManagedObject

- A generic container object that provides efficient storage for the properties defined by its associated `NSEntityDescription` instance

- Associated with

  - A **managed object context** that tracks changes to the object graph

  - An `NSEntityDescription`

NSEntityDescription

- Typically defined in an application's **Managed Object Model**
- Provides metadata about the object, including
  - The name of the entity that the object represents
  - The names of its attributes and relationships

NSManagedObjectContext

- An object space used to manipulate and track changes to **managed objects**

- Provides the means to
  - Retrieve or **fetch** objects from a persistent store
  - Make changes to those objects
  - Delete those objects
  - Insert new objects
  - Commit all changes back to the persistent store

NSFetchRequest

- A description of search criteria used to retrieve and optionally sort a group of managed objects held in a persistent store
- Most commonly contains
  - An entity description
  - A predicate specifying which properties to filter by
  - An array of sort descriptors that determine how the returned objects should be ordered
- See NSFetchRequest documentation for more aspects available to customizing fetch requests